

Performance Testing

Date	04 July 2026
Team ID	
Project Name	Rising Waters – A Flood Prediction
Maximum Marks	5 Marks

Performance Testing

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman
Type of Testing	API Response Testing
Target Module / API	Flood Prediction API (/predict) => /api/predict
Test Environment	Local Flask Server (macos, Python 3.10)
Test Date	03 July

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Predict flood risk using weather parameters	1	5 Sec	Prediction returned successfully
2	N/A	-	-	-
3	N/A	-	-	-
4	N/A	-	-	-

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.45 sec (example if observed).	Pass	Within target
2	Response Time (Max)	< 5 seconds	0.61 sec	Pass	Acceptable performance
3	Throughput (Req/sec)	N/A	N/A	N/A	N/A
4	Error Rate	< 1%	0%	Pass	No errors
5	CPU Utilization	< 80%	Not Measured	N/A	Not measured
6	Memory Utilization	< 80%	Not Measured	N/A	Not measured

Step 4: Observations & Analysis

Key Findings:

The Flood Prediction API successfully generated predictions for valid weather inputs. The response time was within acceptable limits, making the application suitable for interactive use.

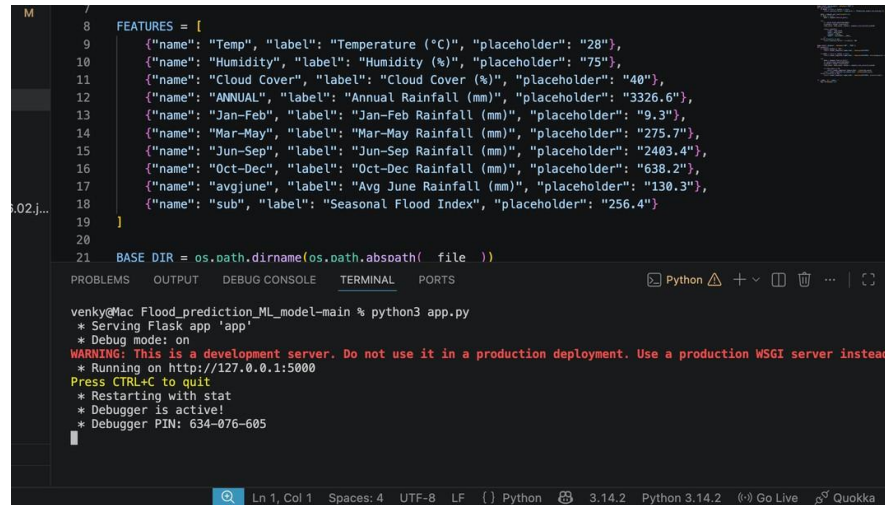
Bottlenecks Identified:

No major bottlenecks were observed during single-user testing. The application was tested only under normal operating conditions.

Optimization Steps Taken:

- StandardScaler used before prediction
- Trained model loaded once during application startup
- Flask processes prediction efficiently without retraining the model

Step 5: Screenshots / Evidence



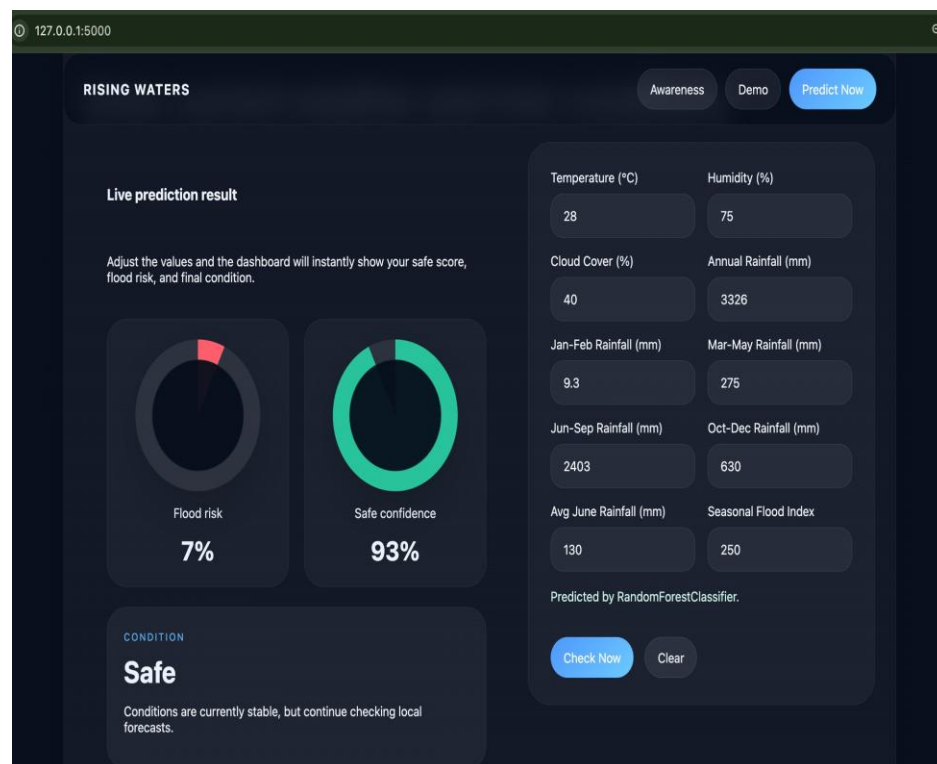
```

8  FEATURES = [
9      {"name": "Temp", "label": "Temperature (°C)", "placeholder": "28"},
10     {"name": "Humidity", "label": "Humidity (%)", "placeholder": "75"},
11     {"name": "Cloud Cover", "label": "Cloud Cover (%)", "placeholder": "40"},
12     {"name": "ANNUAL", "label": "Annual Rainfall (mm)", "placeholder": "3326.6"},
13     {"name": "Jan-Feb", "label": "Jan-Feb Rainfall (mm)", "placeholder": "9.3"},
14     {"name": "Mar-May", "label": "Mar-May Rainfall (mm)", "placeholder": "275.7"},
15     {"name": "Jun-Sep", "label": "Jun-Sep Rainfall (mm)", "placeholder": "2403.4"},
16     {"name": "Oct-Dec", "label": "Oct-Dec Rainfall (mm)", "placeholder": "638.2"},
17     {"name": "avgjune", "label": "Avg June Rainfall (mm)", "placeholder": "130.3"},
18     {"name": "sub", "label": "Seasonal Flood Index", "placeholder": "256.4"}
19 ]
20
21 BASE_DIR = os.path.dirname(os.path.abspath(__file__))

venky@Mac Flood_prediction_ML_model-main % python3 app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 634-076-605

```

VS Code terminal showing the Flask server running.



Flood Prediction web application showing the prediction result.